

How Resona Was Built, Explained Simply

A plain-language walkthrough of the whole project, for technical and non-technical readers alike.

Goal of this document: explain the entire project, from the first idea to the finished, live website, so that both a curious kid and a hiring manager who has never written code could read it and understand exactly what was built, and why every decision was made. No unexplained jargon. Every technical term gets a plain-English definition the first time it shows up, and again in the glossary at the end.

Part 1: What is this thing, in one paragraph?

Resona is a website where you tell a computer program about a song you want to promote, and it does the job of a small marketing team by itself. It watches a sample video to understand its vibe, picks the best people to help spread the song, writes them personalized messages, and shows its work the whole time so you can watch it think, live, in front of you. Nothing about it is faked or pre-recorded. Every time you run it, it is doing real work, in real time.

Part 2: Why does this exist? (The story)

This was built to accompany a job application for an AI Engineer role at a company called Influur. Influur builds an AI product called Pulse that helps music labels and artist managers find the right TikTok creators to promote a song, score them by how well they will actually make the song travel (not just by follower count), write outreach messages to them automatically, and keep a campaign moving even if a creator does not respond.

Rather than writing a resume that just claims a set of skills, the idea was to build a small, working version of the same kind of system, so the skills could be proven instead of asserted. Think of it like a chef auditioning by actually cooking a dish instead of describing one they cooked once.

Resona is original branding. It is not affiliated with Influur, does not use their name "Pulse," and does not use their logo. Every page of the demo says so clearly in the footer.

Part 3: The plan (before any code was written)

3a. Researching the company

Before writing a single line of code, real research went into understanding what Influur's product actually does: it looks at video content and scores creators by how well they fit a specific song's style, not by raw follower count. It writes outreach messages automatically. It can find backup creators if someone does not respond. It treats video content itself, the storytelling style, the pacing, the energy, as a real signal, not just hashtags or demographics.

3b. Deciding what to build (and what not to build)

Two versions of this project were designed on paper: a big, complete version with every feature imaginable, and a smaller version that proves the same underlying skills without handing a company being applied to its entire possible system design before a real conversation ever happens. The smaller version is what got built. Think of it like showing a confident, finished sketch instead of your entire sketchbook, there is more where that came from, but it is not all on the table yet.

3c. Naming it

The name "Resona" comes from resonance, the idea of something ringing true and carrying, which fits a song traveling and reaching people. It deliberately avoids Influur's own product name "Pulse," and the visual design was built from an original color and type direction rather than copying Influur's actual brand assets, even after being asked to more than once during the build. That boundary held throughout the project: inspiration from a company's visual mood is fair game, reproducing their actual logo or trademark is not, especially in something being sent directly to that company.

Part 4: The building blocks (technical decisions, explained for everyone)

For each item below: what it is in plain English, why it was picked, and what would have happened with a different choice.

4a. "Agents": what does that even mean?

An AI agent is like a helpful coworker who can use tools, not just answer questions. Instead of only chatting, it can look things up, do a calculation, or write a message, and then decide what to do next based on the result. "Multi-agent" means a small team of these workers, each with one job, managed by a lead agent (called the orchestrator) that decides which worker to call and when. Resona's orchestrator has three workers on call: one that watches and understands video, one that searches and scores creators, and one that drafts outreach messages.

4b. Why more than one AI model?

Different AI models are good at different jobs, the same way you would not hire a translator to do your taxes. Resona uses a stronger, pricier model for the hard reasoning work: deciding what to do next, and looking at video frames to understand style. It uses a smaller, cheaper, faster model to write several short outreach messages in parallel, since that is a simpler task done many times over. A third,

specialized model exists only to turn text into a list of numbers for comparison (see the next section). Picking the right-sized tool for each job kept the whole system faster and cheaper without losing quality where it actually mattered.

4c. What is "RAG" and "embeddings"? (in plain English)

An embedding is a way of turning a description into a list of numbers that captures its meaning, a bit like a GPS coordinate, but for ideas instead of places. Two similar ideas end up with numbers that are close together. That lets a computer measure how similar two things are, mathematically, without a human having to compare them by hand.

RAG stands for Retrieval-Augmented Generation. In plain terms: look up the most relevant facts first, then let the AI use them to answer, instead of the AI just guessing from memory. Resona turns your campaign brief into an embedding, then compares it against a roster of creator profiles (also turned into embeddings ahead of time) to find the closest matches. No database was needed for this at the current scale, thirty three creator profiles comfortably fit in memory and get compared directly, which is a real engineering call explained more in section 4g.

4d. How did it understand a TikTok video?

When you paste a public TikTok link, the program fetches the video's official preview information and thumbnail image, no login or scraping involved, and hands that to an AI model that can genuinely see images. That model describes the video's storytelling pattern, its pacing (fast cuts or slow?), its energy (high or chill?), and its overall aesthetic, the same way a music supervisor would describe a scene to a colleague. That description becomes a set of structured tags used later in the matching step. If the video cannot be read (a private video, a broken link), the system says so honestly and keeps going using the rest of your brief, instead of pretending or crashing.

4e. How did it score and pick creators?

The final score is not an AI's opinion, it is a plain, visible math formula: audience fit, style match, sound engagement, engagement quality, and reliability, each weighted and added up into one number you can see broken down piece by piece on screen. That formula deliberately never includes raw follower count, a smaller, more engaged creator can and does outrank a much bigger, less-engaged one, which mirrors Influor's own stated approach. Doing the scoring with math instead of asking an AI to eyeball a number keeps it predictable, explainable, and cheap, arithmetic is the wrong job for a language model.

During testing, the original weighting let a handful of "generalist" creators with strong all-around stats win too often, even when the campaign brief changed a lot. The weights were rebalanced to lean harder on the parts of the score that actually depend on your specific brief (audience fit and style match), and lighter on the parts that describe a creator's general track record. That fix was verified live, with real before-and-after numbers, not just assumed to work.

4f. Where does it run, and how does someone see it live?

The website is hosted on Vercel, a service that is like renting a storefront on the internet, so anyone with the link can visit instantly without anything being installed. The code itself lives in a public GitHub

repository, a folder online with a saved history of every change ever made to it, so anyone can see exactly how it was built, not just the finished result.

4g. What did we deliberately leave out, and why?

- **A real database.** At thirty three creator profiles, comparing them directly in memory is faster and simpler than standing up a database, and a full production-ready database design was made on paper but not built into this public demo, on purpose. It is a real, deliberate engineering choice about matching the tool to the actual scale of the problem, not a shortcut.
- **Cross-visit memory.** The system remembers everything within a single run, but starts fresh each time you load the page. A version with saved campaign history across visits is designed but intentionally not part of this scoped-down build.
- **A "replace this creator" feature.** Influur's real product can find a backup creator automatically if someone does not respond. That workflow exists on paper for a fuller version of this project, not in the public demo.
- **Sending anything for real.** Every outreach message drafted by Resona is clearly labeled "draft, never sent." Nothing in this demo contacts a real person.

Part 5: Building it, what actually happened, step by step

1. Researched Influur's actual product and the job description in detail before writing any code.
2. Designed a full, ambitious system on paper first, then deliberately scoped a smaller version to actually build, and wrote down the reasoning for every feature that was cut, so nothing was forgotten, just saved.
3. Built the project's skeleton: the website framework, the styling system, the basic page layout.
4. Wrote thirty three synthetic (made-up but realistic) TikTok creator profiles, since real Influur data was not available, and generated real number-based embeddings for each one.
5. Built the piece that reads a TikTok video's vibe, tested against real, live public videos, not fake ones.
6. Built the matching and scoring math, with its own automated tests proving it behaves correctly, including a test that specifically proves a smaller, more engaged creator can beat a much bigger, disengaged one.
7. Built the message-writing piece and connected all three pieces into one orchestrator that runs them in order and streams its progress live to the screen.
8. Built the results page: a live trace of what the AI is doing, a ranked shortlist with a visible score breakdown per creator, and the drafted outreach messages.
9. Tested the whole flow end to end, including on purpose feeding it a broken video link and confirming it recovered gracefully instead of crashing.
10. Put the code on a public GitHub repository, checked carefully first to make sure no private planning documents or secrets accidentally went with it.

11. Deployed the website live on the internet, then found and fixed a real access problem, the hosting service had accidentally left a login wall up that would have blocked anyone outside the project owner's account, including a recruiter clicking the link cold.
 12. After the initial launch, ran several more rounds of visual and interaction polish: a live animated background, smoother scrolling, more considered typography and color, and a two-column layout for the results view, each round checked live in a real browser before being shipped, not just assumed to look right.
 13. Ran a dedicated pressure-testing pass specifically because this was going out for a real job application: tried to break the form with bad input, double submissions, page reloads mid-run, and keyboard-only navigation. Found one real bug this way (a budget field silently rejecting almost any value a real person would type), fixed it, and re-verified.
 14. Investigated a report that the same creators kept winning regardless of input, traced it to a real scoring imbalance, fixed the weighting, and proved the fix with real before-and-after score math, not just a claim that it was fixed.
 15. Promoted the final, tested build to the one permanent public link.
-

Part 6: Testing (how we know it actually works)

QA, short for quality assurance, means pretending to be a real visitor and deliberately clicking through everything before a real person, like a recruiter, ever sees it. An automated browser-testing tool was used to actually load the live website and interact with it the way a real visitor would, not just read the code and assume it works.

What got specifically checked and confirmed:

- A full real run, start to finish, with a real TikTok video and real generated results.
- A broken or non-TikTok video link, confirming the system explains the problem clearly and still finishes the rest of the job.
- Required form fields correctly blocking submission when left empty.
- Clicking submit twice quickly, confirming only one campaign runs, not two.
- Reloading the page in the middle of a run, confirming it recovers cleanly instead of getting stuck.
- The full flow on a phone-sized screen.
- Using only a keyboard, no mouse, confirming every field and the submit button are reachable and usable in order.
- Deliberately extreme input, including a script tag typed into a text field, confirming it is safely ignored rather than run.
- A budget value that was not a round number, which is how the one real bug on this list was actually found.

- Zero errors in the browser's own error console across every one of the above, checked directly, not assumed.
-

Part 7: What's next, and what's saved for later

There is a larger, more complete version of this system designed on paper, with more advanced features like cross-visit memory, an automatic creator-replacement flow, and a production-scale database. That fuller design is intentionally not published here. It is being saved to walk through directly in a real conversation, rather than handed over in a public repository before one ever happens.

Glossary (plain-English definitions of every technical term used above)

- **Agent:** an AI system that can take actions, not just answer questions, by calling tools to look things up or do tasks.
- **API:** a way for two computer programs to talk to each other, like a waiter carrying an order between a customer and a kitchen.
- **Cosine similarity:** the specific math used to measure how close two embeddings are to each other, the closer, the more similar the underlying ideas.
- **Deterministic:** producing the exact same output every time for the exact same input, like a calculator, as opposed to an AI that might phrase things differently each time even when asked the same thing.
- **Embedding:** a way of turning words or ideas into a list of numbers so a computer can measure how similar two things are.
- **LLM (Large Language Model):** the technology behind AI chatbots like Claude or ChatGPT, a program trained to understand and generate human language.
- **Orchestrator:** the lead AI agent in a multi-agent system, responsible for deciding which worker agent or tool to use next.
- **RAG (Retrieval-Augmented Generation):** looking up relevant information first, then having the AI use it to respond, instead of relying only on what it already knows.
- **Repository ("repo"):** a folder of code with a saved history of every change, usually hosted on a site like GitHub.
- **Vector / vector search:** another name for an embedding and the act of searching for the closest matches to one.

Resona is a portfolio demo built for Influur's AI Engineer role. It is not affiliated with Influur.